

Teaching AI the Way We Teach Data Science

Open, hands-on, built to scale

Eric Van Dusen

College of Computing, Data Science, and Society, UC Berkeley

October 9, 2026

UC Berkeley

UC Berkeley College of Computing,
Data Science, and Society

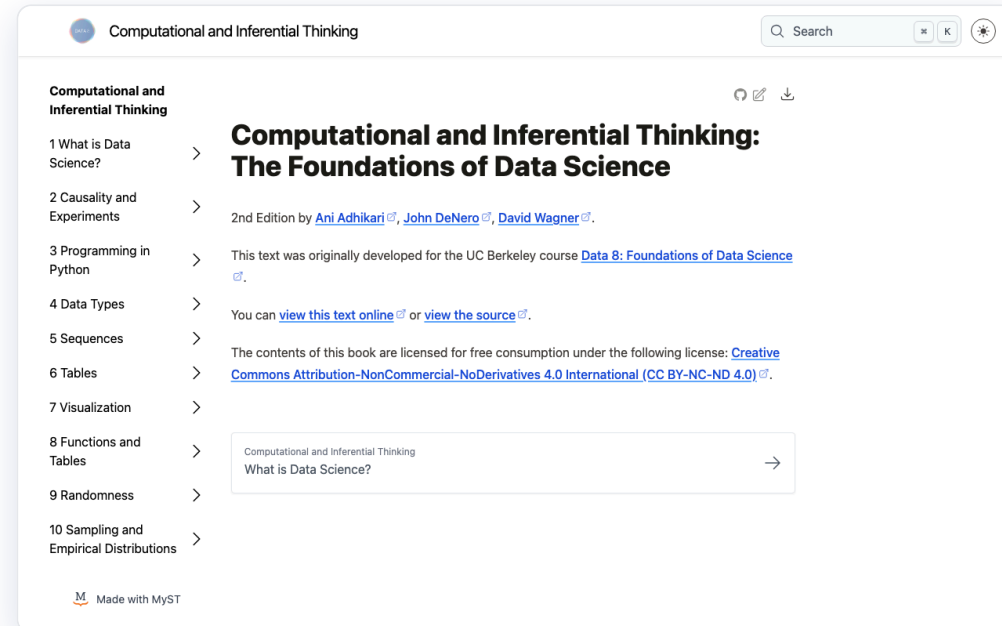
The Data 8 Story



1. WHAT DO WE KNOW WORKS?

A first course in computing and statistics, for any major, in the browser, with real data from day one.

Largest course at UC Berkeley - up to 50% of undergraduates take it, and the gateway to Berkeley's biggest major.



inferentialthinking.com

We didn't make students learn all the machinery before we let them do data science.

The notebook became the laboratory

1. WHAT DO WE KNOW WORKS?



- Jupyter + Python + real data
- Change code, immediately see what happens
- Same environment for instructors and students
- Reproducible across a class
- Works for 20 students or thousands
- Open-source tools + open curriculum + shared infrastructure

Three lessons from a decade of scaling

1. WHAT DO WE KNOW WORKS?

Remove friction

JupyterHub means no installs and no laptop gap. Students meet ideas before they fight environments.

Connect to disciplines

Connector courses (like Data 88E) provide domain context that makes methods stick.

Open everything

Github Deployment, Open Stack of tools, Open textbook, notebooks, and autograding are what let it travel.

Scale is more than course materials

1. WHAT DO WE KNOW WORKS? HOW TO GET 25 COMMUNITY COLLEGES TO ADOPT IT?

- **Infrastructure:** shared JupyterHubs through for less resourced institutions (eg NSF Cloudbank or Cal-ICOR)
- **Curriculum:** open materials community colleges can adopt and adapt
- **People:** faculty networks, workshops training, and credit that transfers

How much of this can we do for AI education?

What if we did the same thing with AI?

2. CAN WE ADAPT WHAT WE TEACH?

Put a model in the notebook.

Prompt it

Change parameters

Examine tokens and probabilities

Create embeddings

Add retrieval

Compare models

Build evaluations

Investigate failures

The notebook can become a laboratory for AI.

Who owns the laboratory?

2. CAN WE ADAPT WHAT WE TEACH?

Commercial AI

Notebook



API



Proprietary model



Commercial cloud

Student learns to **consume** AI

Open AI

Notebook



Open-source tooling



Open model



Shared / public compute

Student can **inspect, modify, evaluate and build**

Small Open Models as teaching tools

2. CAN WE ADAPT WHAT WE TEACH?

You don't need a frontier model for every experiment. A small open model is just a file students can count, inspect, and test.

- Parameters $\times$ bits $\div$ 8 = size
- Tokens and next-token probabilities
- Temperature: what "randomness" means
- The linear algebra underneath
- "harness" - feed data in, get data out with data manipulation

```
from transformers import AutoTokenizer, AutoModelForCausalLM

name = "HuggingFaceTB/SmolLM2-135M" # a 270 MB file
tok = AutoTokenizer.from_pretrained(name)
lm = AutoModelForCausalLM.from_pretrained(name)

ids = tok("Amsterdam is famous for its", return_tensors="pt")
probs = lm(**ids).logits[0, -1].softmax(-1)

for p, i in zip(*probs.topk(5)):
    print(tok.decode(i), f"{p:.0%}")
```



Small Models in a Jupyter Hub

2. CAN WE ADAPT WHAT WE TEACH?

```
In [10]: # set the model path
model_path = "shared-rw/qwen2-1.5b-instruct-q4_0.gguf"
# example model path, change to your model location
```

```
In [11]: # Load the model from shared storage
model = Llama(
    model_path=model_path,
    n_ctx=2048, # Context window size (how much text the model can process at once)
    verbose=False # Set to True if you want to see detailed loading information
)

print("Model loaded successfully!")
```

Model loaded successfully!

Generating Text

Now let's try generating some text. We'll give the model a prompt and see what it produces.

```
In [12]: # Generate text from a simple prompt
import textwrap

prompt = "Python is a programming language that"

output = model(
    prompt,
    max_tokens=100, # Maximum length of generated text
    temperature=0.7, # Controls randomness (0 = deterministic, 1+ = creative)
    stop=["\n\n"] # Stop generation at double newline
)

# Extract the generated text
generated_text = output['choices'][0]['text']
print(f"Prompt: {prompt}")
print(f"Generated:\n{textwrap.fill(generated_text, width=80)}")
```

Prompt: Python is a programming language that
Generated: allows you to do many different things. It can be used for web development, data analysis, machine learning, and more. It's also a great language for beginners because it's easy to learn and has a lot of great resources online to help you get started. Python is also widely used in scientific research, as it's a powerful tool for data analysis and machine learning.

The model downloads into a shared location on *JupyterHub* and runs in browser. Students can inspect the model, change parameters, and see what happens, e.g. with no setup.

Admin sets up environment, prof sets up curriculum, student explores.

[Cloudbank Classroom](https://cloudbank.org/training/access-cloudbank-classroom): cloudbank.org/training/access-cloudbank-classroom

Small Models in the Browser

2. CAN WE ADAPT WHAT WE TEACH?



Try it now
jupyterlite.github.io/ai



The whole notebook, and the model, runs on your own laptop. **No account. No server. No cost.**

- **jupyterlite-ai**: chat and code completion inside JupyterLite
- **Pyodide**: Python compiled to WebAssembly
- **WebLLM**: LLMs on your GPU through WebGPU
- **Transformers.js**: Hugging Face models in the browser
- **willama**: llama.cpp compiled to WebAssembly

Thanks to Jeremy Tuloup!

From algorithms to systems

2. CAN WE ADAPT WHAT WE TEACH?

What counts as technical literacy?

Traditional computing / ML

Programming



Mathematics



Data structures



Algorithms



Machine learning

Modern AI workflow

Data



Models



APIs



Containers



Cloud



Deployment



Evaluation



Monitoring

Students need to see the whole stack

2. CAN WE ADAPT WHAT WE TEACH?

A student can understand **gradient descent** but have no idea how an AI application actually works.

A student can now build an **AI application** without understanding gradient descent.

Both are educational problems.

- Data engineering
- APIs
- Containers and reproducible environments
- Cloud / GPUs / inference
- MLOps / LLMOps
- Evaluation and monitoring

Teach enough engineering that the system stops looking like magic.

Same notebook, three rungs

3. WHAT INVESTMENTS DO WE NEED?

1

Browser

JUPYTERLITE

Small models run on the student's own laptop. Free, private, zero setup.

2

Campus hub

JUPYTERHUB

Shared model folders, curated environments, GPUs when a course needs them.

3

HPC and cloud slices

JUPYTER AS THE FRONT DOOR

A small allocation for every student who needs one. Powerful but hard to use today, so **invest in making it easy**.

Teach the stack along the way: environments, containers, git, job schedulers, cloud costs. DevOps is part of AI literacy.

The model question

3. WHAT INVESTMENTS DO WE NEED?

- Today's small open models come mostly from a few labs, mostly China
- Corporate openness can change with the next release (e.g. OpenAI, Google, Meta)
- Build open curriculum that is flexible enough to swap models in and out, but worth having a small open model that is always available for teaching

Small + Open + Public

3. WHAT INVESTMENTS DO WE NEED?

Europe doesn't have to win the race for the **biggest model**.

It can compete differently, and it already has pieces at every rung.

Plus a mandate: AI literacy under Article 4 of the EU AI Act

Rung	European pieces
Browser	QuantStack, JupyterLite
Campus hub	SURF and national research infrastructure
Public compute	EuroHPC
Models	GPT-NL, OpenEuroLLM

Can Europe build a lighter-weight, open AI ecosystem rather than simply reproducing the hyperscalers?

What should the university build?

3. WHAT INVESTMENTS DO WE NEED?

Students across the disciplines

Open curriculum

Interactive notebooks

Open-source tooling

Open models

Public / shared compute

Goal 1: shared open **curriculum** that runs at multiple levels and can move between universities.

Goal 2: a small, fully open, multilingual **teaching model** with documented data.

Data Science to AI

Interactive pedagogy

We know how to make technical ideas accessible through **exploration**.

Systems literacy

Students need to understand **workflows**, not only algorithms.

Open infrastructure

Universities should help students become **builders**, not simply customers.

We are still scaling data science. Next: give everyone the ability to look inside AI.

For the panel: who should own the infrastructure that teaches the next generation what AI is?

ericvd@berkeley.edu